Access Mode Project Guide

(V1.0-Access-mode)

Document Control Information

Document Identifier: AMPG-ORCH-001
Title: Access Mode Project Guide
Subtitle: Practical Project Setup and Reasoning Discipline
Version: V1.0-Access-Mode
Date of Issue: 5th February 2026
Status: Canonical Orchestration Guide (Non-Normative)
Author: Anthony Siljeg
Affiliation: Alnotrea Labs / Policy of Developments — Access PoD *Initiative*

Introduction

This document explains how to create and use **AI projects** based on **Access-mode**.

Access-mode is a way to control *how* an AI system reasons by:

- selecting a single primary reasoning mode,
- optionally allowing limited secondary support modes, and
- enforcing clear boundaries through project structure.

This guide focuses on **project setup and usage**, not AI model internals.
It is **model-agnostic** and can be used with any large language model that has access to the correct documents and datasets.

The document is written for people who want:

- consistent explanations,
- reduced hallucination and over-confidence,
- clearer separation between learning, mentoring, and organizing tasks.

This guide does **not** define compliance, certification, or deployment readiness.
All outputs remain drafts and require human judgment.

IMPORTANT — Canonical Orchestration Guide

Access_Mode_Project_Guide.PDF is the authoritative reference for:

- creating AI projects using Access-Mode, and
- applying mode discipline (single-mode and mixed-mode).

The following documents are supporting references only:

- Access-Mode Quick Reference — concise summary for facilitators and reviewers
- Facilitator Quick-Start Card — workshop delivery guidance
- Access-Mode Index — standards-facing orientation and lifecycle context

Where guidance differs between documents, this guide takes precedence.

Table of Contents

1. PURPOSE AND SCOPE 5
Purpose 5
Scope 5
Out of Scope 5
Intended Audience 6
Reading Guidance 6
2. WHAT ACCESS-MODE IS (AND IS NOT) 7
What Access-Mode Is 7
What Access-Mode Is Not 7
Why Access-Mode Exists 8
Access-Mode vs Prompting Alone 8
Access-Mode as a Project Boundary 9
Key Principle 9
3. THE THREE ACCESS-MODES (ID_1, ID_2, ID_3) 10
Overview 10
3.1 Mode ID_1 — Core (Foundational Reasoning) 10
3.2 Mode ID_2 — Access-Mode (Operational / Runtime Reasoning) 11
3.3 Mode ID_3 — Operator / Standards (Inspection Reasoning) 12
3.4 Key Differences at a Glance 13
3.5 One Primary Mode Rule 13
4. MODE DISCIPLINE AND CROSS-MODE RULES 15
Purpose of Mode Discipline 15
4.1 One Primary Mode Rule 15
4.2 Secondary Modes (Limited Support Only) 15
4.3 Allowed Cross-Mode Patterns 16

4.4 Prohibited Cross-Mode Behavior ... 17
4.5 organizer_mode.md (Control File) ... 17
4.6 Human Responsibility and Escalation ... 18
Key Principle ... 18
5. VALIDATION, QUESTIONING, AND RECORD-KEEPING ... 19
Purpose of Validation ... 19
5.1 Validation Is Mandatory ... 19
5.2 Misalignment Testing (Required Step) ... 19
5.3 Question-Driven Verification ... 20
5.4 Mode-Aware Validation ... 21
5.5 Record-Keeping (Lightweight, Required) ... 21
5.6 When to Stop or Reset ... 21
Key Principle ... 22
6. CREATING AI PROJECTS USING ACCESS-MODE ... 23
PURPOSE OF PROJECT SETUP ... 23
ACCESS-MODE PROJECT CONFIGURATION (SIMPLIFIED) ... 23
Single-mode project (default) — 8 files ... 23
Single-mode project with restricted settings — 8 files ... 24
Mixed-mode project (advanced) — 14 files ... 24
GOVERNING RULE ... 24
6.1 ONE PROJECT, ONE PURPOSE ... 25
6.2 CREATING A NEW PROJECT ... 25
6.3 SELECTING THE PRIMARY ACCESS-MODE ... 25
6.4 UPLOADING REQUIRED FILES (SINGLE-MODE) ... 26
6.5 PROJECT NAMING (CONTROL SIGNAL) ... 26
6.6 WHEN TO USE MIXED-MODE PROJECTS ... 27
6.7 STABILITY RULE ... 27
KEY PRINCIPLE ... 27
7. SINGLE-MODE PROJECT SETUP ... 28
Purpose of Single-Mode Projects ... 28
7.1 Single-Mode Setup Principles ... 28
7.2 ID_1 — Learning Project (Core Mode) ... 28
7.3 ID_2 — Mentor Project (Operational Mode) ... 29
7.4 ID_3 — Organizer Project (Inspection Mode) ... 30
7.5 How to Detect Single-Mode Drift ... 31
Key Principle ... 32
8. MIXED-MODE PROJECT SETUP AND `ORGANIZER_MODE.MD` ... 33
Purpose of Mixed-Mode Projects ... 33
8.1 When Mixed-Mode Is Appropriate ... 33
8.2 Core Rule (Non-Negotiable) ... 33
8.3 File Requirements (Mixed-Mode) ... 34
8.4 `organizer_mode.md` (Mandatory Control File) ... 34
8.5 What `organizer_mode.md` Must Declare ... 34
8.6 Approved Mixed-Mode Patterns ... 35
8.7 Prohibited Mixed-Mode Patterns ... 36

8.8 Project Naming for Mixed-Mode 36
8.9 How to Operate a Mixed-Mode Session 37
Key Principle 37
9. PROJECT OPENING INSTRUCTIONS (STARTER PROMPTS) 38
Purpose of Opening Instructions 38
9.1 Learning Project — ID_1 (Core Mode) 38
9.2 Mentor Project — ID_2 (Operational Mode) 38
9.3 Organizer Project — ID_3 (Inspection Mode) 39
9.4 Mixed-Mode Projects (Reminder) 39
When to Re-Use an Opening Instruction 39
Key Principle 40
10. WORKING PROMPT EXAMPLES (REFERENCE) 41
Purpose of This Chapter 41
10.1 ID_1 — Learning Prompts (Core Mode) 41
10.2 ID_2 — Mentor Prompts (Operational Mode) 41
10.3 ID_3 — Organizer Prompts (Inspection Mode) 42
10.4 Mixed-Mode Prompts (Controlled Use) 42
10.5 Prompts to Avoid (All Modes) 43
10.6 Using This Chapter Safely 43
Key Principle 43

Formal method definitions for Access-Mode ID_1, ID_2, and ID_3 are provided in APOD-TR-006 — Second Edition, Version E (PDF found in the release folder).

That paper is the canonical reference for standards and review contexts.

1. Purpose and Scope

Purpose

The purpose of this document is to explain **how to create and use AI projects** based on **Access-Mode** in a clear, repeatable, and low-risk way.

Access-Mode is used to control **how an AI system reasons**, not to determine correctness, compliance, or authority. This guide helps users:

- create AI projects with a **single, clear reasoning posture**,
- reduce confusion caused by mixed or implicit modes,
- structure AI interaction around learning, mentoring, or organizing tasks,
- and keep generated content inspectable and reviewable by humans.

This document is written as a **practical guide**, not a specification.

Scope

This guide applies to:

- any large language model (LLM),
- any AI platform that supports document upload or project-based workspaces,
- and any use case where governance-related reasoning, explanation, or organization is required.

The scope of this document includes:

- defining Access-Mode concepts in plain language,
- explaining how to create AI projects using Access-Mode modules,
- describing single-mode and mixed-mode project setups,
- providing starter instructions and prompt patterns,
- and establishing validation and record-keeping discipline.

Out of Scope

This document does **not**:

- define AI system requirements or architectures,
- provide legal, regulatory, or compliance advice,
- certify correctness, safety, or deployment readiness,

- validate AI outputs automatically,
- or replace human expertise and judgment.

Any content generated using this guide remains **draft material** and must be reviewed by a human before use.

Intended Audience

This document is intended for:

- facilitators and educators,
- governance practitioners,
- reviewers and standards participants,
- and individuals using AI as a learning, mentoring, or organizing tool.

The document assumes **no specific technical background**, only that the reader is willing to apply discipline when setting up and using AI projects.

Reading Guidance

This guide is designed to be read **in order**.

Later sections rely on concepts introduced earlier, especially:

- Access-Mode definitions,
- primary vs secondary mode rules,
- and project setup discipline.

If content becomes confusing, return to:

- **Chapter 3 — The Three Access-Modes**, and
- **Chapter 4 — Mode Discipline and Cross-Mode Rules**.

---

2. What Access-Mode Is (and Is Not)

What Access-Mode Is

Access-Mode is a **reasoning control method**.

It defines **how an AI system should think about a task**, not what the final answer should be.

Access-Mode works by:

- selecting a **primary reasoning posture**,
- limiting what assumptions the AI may make, and
- preventing unintentional mixing of different reasoning styles.

In practice, Access-Mode helps:

- reduce hallucination and over-confidence,
- keep explanations consistent,
- make AI outputs easier to review, and
- separate learning, operational thinking, and inspection-oriented work.

Access-Mode is applied through:

- project structure,
- uploaded document sets, and
- explicit instructions.

What Access-Mode Is Not

Access-Mode is **not** an AI capability, feature, or model.

It does **not**:

- improve model intelligence,
- validate correctness,
- guarantee accuracy,
- or make decisions on behalf of humans.

Access-Mode does **not**:

- certify compliance,
- provide legal or regulatory advice,
- assert deployment readiness,

- or replace expert judgment.

It is a **discipline**, not an authority.

Why Access-Mode Exists

Large language models can reason in many ways at once.
Without boundaries, this often leads to:

- mixed explanations,
- hidden assumptions,
- unjustified certainty,
- or answers that shift tone mid-response.

Access-Mode exists to prevent this.

By enforcing a single primary reasoning posture, Access-Mode makes AI behavior:

- more predictable,
- easier to inspect,
- and safer to use in governance-related contexts.

Access-Mode vs Prompting Alone

Prompting alone is often:

- implicit,
- fragile,
- and easy to drift.

Access-Mode is different because it:

- persists across a project,
- applies to all interactions, and
- is reinforced by document selection and structure.

In short:

- prompts guide *what you ask*,
- Access-Mode governs *how the AI responds*.

Access-Mode as a Project Boundary

Access-Mode is applied **at the project level**, not per message.

Once a project is set up:

- the reasoning posture should remain stable,
- secondary perspectives must be explicitly limited,
- and mode switching should require a new project.

This prevents accidental blending of:

- learning language,
- operational reasoning, and
- inspection-oriented framing.

Key Principle

Access-Mode controls reasoning posture, not outcomes.

Human review remains authoritative at all times.

---

3. The Three Access-Modes (ID_1, ID_2, ID_3)

Access-Mode defines **three distinct reasoning postures**.
Each posture is designed for a different type of work.

Only **one Access-Mode should be primary** within a project at any time.

Overview

The three Access-Modes are:

- **ID_1 — Core (Foundational Reasoning)**
- **ID_2 — Access-Mode (Operational / Runtime Reasoning)**
- **ID_3 — Operator / Standards (Inspection Reasoning)**

Each mode:

- has a clear intent,
- permits specific types of reasoning,
- and explicitly restricts others.

They are **not maturity levels** and **not ranked**.

3.1 Mode ID_1 — Core (Foundational Reasoning)

Intent

Mode ID_1 is used to establish **shared understanding**.

It focuses on:

- definitions,
- concepts,
- scope,
- intent, and
- reasoning foundations.

ID_1 avoids assumptions about how systems behave in operation or how artifacts will be inspected.

Use ID_1 when you want to

- learn or teach AI governance concepts,
- align terminology and meaning,
- explain frameworks or structures,
- reason about *what something is* and *why it exists*.

What ID_1 allows

- Conceptual explanations
- Plain-language descriptions
- Clarification of assumptions and scope
- Reframing for understanding

What ID_1 does not allow

- Runtime behavior claims
- Enforcement or telemetry assumptions
- Compliance or inspection conclusions
- Adequacy or sufficiency statements

If such questions arise, they must be deferred to another mode.

3.2 Mode ID_2 — Access-Mode (Operational / Runtime Reasoning)

Intent

Mode ID_2 is used to reason about **governance under operation**.

It focuses on:

- behavior under change,
- stress or failure scenarios,
- drift over time,
- and operational constraints.

ID_2 assumes a system is *in use*, but does not assert that it is safe, correct, or compliant.

Use ID_2 when you want to

- explore what happens when assumptions are tested,
- analyze governance under pressure,
- discuss runtime controls or monitoring concepts,
- examine how failure or misuse might occur.

What ID_2 allows

- Scenario-based reasoning
- Stress and edge-case discussion
- Operational implications
- Exploratory “what-if” analysis

What ID_2 does not allow

- Certification claims
- Statements of compliance or approval
- Final judgments of adequacy
- Replacement of human decision-making

All conclusions remain provisional and reviewable.

3.3 Mode ID_3 — Operator / Standards (Inspection Reasoning)

Intent

Mode ID_3 is used to prepare content for **review or inspection**.

It focuses on:

- clarity,
- structure,
- traceability, and
- interpretation by external reviewers.

ID_3 is concerned with *how content is read*, not *how systems behave*.

Use ID_3 when you want to

- organize governance material,
- refine language for inspection,
- remove ambiguity,
- anticipate reviewer questions.

What ID_3 allows

- Language tightening
- Structural reorganization
- Explicit assumption listing
- Reviewer-oriented framing

What ID_3 does not allow

- Operational reinterpretation
- Runtime conclusions
- Approval or endorsement language
- Compliance assertions

ID_3 improves readability, not authority.

3.4 Key Differences at a Glance

- **ID_1** asks: *What does this mean?*
- **ID_2** asks: *What happens under pressure?*
- **ID_3** asks: *How will this be read or reviewed?*

Each question requires a different posture.

3.5 One Primary Mode Rule

At any time:

- **one mode is primary**, and
- any other mode may only provide limited support.

Primary mode determines:

- tone,
- assumptions,
- and allowed conclusions.

Switching primary modes requires:

- ending the current session, or
- creating a new project.

The formal method definitions of ID_1, ID_2, and ID_3 are provided in Version E (PDF) for standards and review contexts

---

4. Mode Discipline and Cross-Mode Rules

Purpose of Mode Discipline

Mode discipline exists to prevent **unintentional mixing of reasoning styles**.

Large language models can shift tone, assumptions, and authority very easily.
Without discipline, this leads to:

- contradictory answers,
- hidden assumptions,
- overconfident conclusions,
- and outputs that are difficult to review or trust.

Mode discipline ensures that:

- the AI stays within a **defined reasoning posture**, and
- humans can clearly interpret *why* an answer was produced in a certain way.

4.1 One Primary Mode Rule

At any time, **one Access-Mode must be primary**.

The primary mode determines:

- what assumptions are allowed,
- what conclusions are permitted,
- and how certainty is expressed.

The primary mode is set by:

- the **project setup**,
- the **uploaded module set**, and
- the **opening instruction**.

If the primary mode is unclear, the project is considered **misconfigured**.

4.2 Secondary Modes (Limited Support Only)

In some projects, a **secondary mode** may be allowed.

Secondary modes:

- provide perspective,
- offer clarification,
- or help reframe content.

Secondary modes **do not**:

- introduce new authority,
- change the reasoning posture,
- or override the primary mode.

Secondary modes are optional and must be **explicitly declared**.

4.3 Allowed Cross-Mode Patterns

The following cross-mode patterns are permitted when **explicitly defined**.

ID_1 Primary → ID_2 Secondary

(Learning with operational perspective)

Allowed:

- identifying where assumptions may fail,
- discussing potential stress points.

Not allowed:

- runtime adequacy claims,
- enforcement conclusions.

ID_1 Primary → ID_3 Secondary

(Learning with inspection clarity)

Allowed:

- clearer wording,
- reduced ambiguity,
- reviewer-friendly structure.

Not allowed:

- standards conformance claims,
- approval or certification language.

ID_2 Primary → ID_3 Secondary

(Operational reasoning with inspection framing)

Allowed:

- organizing operational reasoning,
- preparing review notes.

Not allowed:

- reinterpreting runtime behavior,
- downgrading uncertainty.

4.4 Prohibited Cross-Mode Behavior

The following are **not allowed**:

- multiple primary modes at the same time,
- switching primary modes mid-session,
- blending learning, operational, and inspection language in one answer,
- using inspection language to justify operational adequacy,
- implying compliance, certification, or approval.

If a request requires a different primary mode:

- stop the session, or
- switch to a different project.

4.5 organizer_mode.md (Control File)

For mixed-mode projects, an **organizer_mode.md** file is required.

This file:

- explicitly declares the **primary mode**,
- lists any **allowed secondary modes**,
- and defines strict boundaries.

If organizer_mode.md is missing:

- the project must be treated as **single-mode only**.

If organizer_mode.md is unclear or contradictory:

- outputs should be considered unreliable.

4.6 Human Responsibility and Escalation

Mode discipline does **not** remove human responsibility.

Humans must:

- monitor posture drift,
- correct misalignment,
- and decide when to stop or reset a session.

If the AI produces content outside the defined mode:

- this should be recorded,
- corrected,
- and used as a learning signal.

Key Principle

One project, one primary mode.
Secondary modes assist — they do not decide.

---

5. Validation, Questioning, and Record-Keeping

Purpose of Validation

AI outputs must **never be accepted by default**.

Even when mode discipline is correct, large language models can:

- omit constraints,
- infer missing information,
- or sound confident while being incomplete or wrong.

Validation ensures that outputs are:

- grounded in the uploaded material,
- consistent with the selected Access-Mode, and
- suitable for human review.

Validation is a **process**, not a single check.

5.1 Validation Is Mandatory

For every meaningful output, validation **must occur**.

Validation is required when:

- content will be reused,
- content will be shared with others,
- content informs decisions,
- or content is documented or archived.

Skipping validation increases the risk of:

- silent errors,
- misinterpretation,
- and false confidence.

5.2 Misalignment Testing (Required Step)

Misalignment testing is the **primary validation method**.

How misalignment testing works

After generating content:

1. **Remove or alter** two or three elements, such as:
    - a definition,
    - a scope boundary,
    - a key assumption,
    - or a reference.
2. **Re-run the same model** with the altered input.
3. **Observe the response**.

What to look for

- Does the model detect the inconsistency?
- Does it propagate the error?
- Does it invent missing structure?

If the model:

- confidently fills gaps without evidence, or
- ignores contradictions,

the output is **not reliable** without correction.

5.3 Question-Driven Verification

Misalignment testing should be paired with **direct questioning**.

Ask questions that force the model to expose assumptions.

Examples:

- “Which parts of this answer are inferred rather than stated?”
- “What assumptions are not directly supported by the modules?”
- “What would a reviewer likely challenge here?”
- “Which Access-Mode constraints are shaping this answer?”

Do not accept a single answer as final.

5.4 Mode-Aware Validation

Validation must respect the **active Access-Mode**.

- In **ID_1 (Learning)**:
  - Check clarity, scope, and definition accuracy.
- In **ID_2 (Operational)**:
  - Check scenario realism and uncertainty handling.
- In **ID_3 (Organizer / Standards)**:
  - Check language precision and interpretability.

Validation failure often indicates **mode drift**, not model failure.

5.5 Record-Keeping (Lightweight, Required)

Keep **brief records** of important sessions.

Records do not need to be complex.

For each session, note:

- the project name,
- the primary Access-Mode,
- any secondary modes used,
- validation actions taken,
- issues found,
- and corrections made.

This can be:

- a short text note,
- a markdown file,
- or a simple log entry.

5.6 When to Stop or Reset

Stop the session if:

- the model repeatedly ignores constraints,
- mode drift cannot be corrected,
- or validation consistently fails.

If a different reasoning posture is required:

- end the session, and
- switch to a different project.

Do **not** force a posture change inside the same project.

Key Principle

Validation confirms grounding.
Questioning exposes assumptions.
Record-keeping preserves clarity.

Human judgment remains authoritative at all times.

---

6. Creating AI Projects Using Access-Mode

Purpose of Project Setup

Access-Mode is applied at the **project level**.

A project acts as a *reasoning container* that defines:

- the primary Access-Mode,
- which documents the AI system may rely on, and
- how all outputs must be interpreted.

Correct project setup is the **most important control** in this guide.
Most failures in AI-assisted governance occur due to **incorrect or incomplete setup**, not due to poor prompts.

Once a project is created and files are uploaded, the reasoning posture is fixed for the duration of the session.

Canonical Configuration Note — Access-Mode Projects

Every Access-Mode project must include a complete and coherent file set.

Each Access-Mode (ID_1, ID_2, ID_3) consists of:

- a mode-specific README file,
- a mode-specific control file, and
- six modules that define the reasoning surface.

These files take precedence over prompts or ad-hoc instructions.
Partial uploads, missing files, or mixed file sets invalidate the project configuration.

Access-Mode Project Configuration (Simplified)

Single-mode project (default) — 8 files

- One mode-specific README file
- One mode-specific control file
- Six modules from the *same* Access-Mode

Rules:

- All eight files **must** be uploaded together.
- No modules from other modes may be included.
- No project-specific organizer file is used.
- The project remains strictly single-mode.

Single-mode project with restricted settings — 8 files

- One mode-specific README file
- One mode-specific control file (used to further restrict or clarify behavior)
- Six modules from the same Access-Mode

Rules:

- The project remains single-mode.
- The control file may only *restrict*, never expand, reasoning permissions.

Mixed-mode project (advanced) — 14 files

- One mode-specific README file (primary mode)
- Six modules from the **primary** Access-Mode
- Six modules from **one** secondary Access-Mode
- One project-specific control file: **organizer_mode.md** (mandatory)

Rules:

- Primary vs secondary authority must be explicitly declared.
- Secondary mode may assist but must never decide.
- Mixed-mode projects without organizer_mode.md are invalid.

Governing Rule

- **Single-mode projects never introduce mixed-mode authority.**
- **Mixed-mode projects always require an explicit project-level control file.**

If any required file is missing, the project is misconfigured and outputs must not be relied upon.

6.1 One Project, One Purpose

Each project must have **one clear purpose**.

A project should be created for:

- learning,
- mentoring (operational reasoning), or
- organizing / reviewing.

Do not attempt to cover multiple purposes in a single project unless mixed-mode rules are explicitly applied (see Chapter 8).

If the project purpose is unclear, reasoning posture will drift.

6.2 Creating a New Project

To create a new project:

1. Create a new, empty project in your AI platform.
2. Do not reuse an existing project.
3. Do not remove, replace, or swap files mid-session.

Once files are uploaded and the session begins, the project must be treated as **stable**.

6.3 Selecting the Primary Access-Mode

Before uploading any files, decide which Access-Mode will be primary.

Choose **one**:

- **ID_1 — Core (Learning)**
  For explanation, understanding, and conceptual clarity.
- **ID_2 — Operational (Mentor)**
  For reasoning about runtime behavior, stress, and change.
- **ID_3 — Organizer / Standards**
  For structuring, reviewing, and inspection-oriented work.

This decision controls everything that follows.

6.4 Uploading Required Files (Single-Mode)

For a single-mode project, upload **all required files for one Access-Mode only**:

- the mode-specific README file,
- the mode-specific control file, and
- all six modules for that mode.

Examples:

- Upload **all files** from `ID_1_core/`
- OR **all files** from `ID_2_access/`
- OR **all files** from `ID_3_operator/`

Rules:

- Do not upload partial sets.
- Do not mix modules from different modes.
- Do not upload canonical `/modules/` source files directly.

The uploaded files together become the **authoritative reference base** for the project.

6.5 Project Naming (Control Signal)

The project name signals how the AI system should behave.

Use simple, literal titles.

Recommended patterns:

- *My AI Governance — Learning (ID_1)*
- *My AI Governance — Mentor (ID_2)*
- *My AI Governance — Organizer (ID_3)*

Avoid words such as:

- expert,
- authority,
- validator,
- certifier.

These increase the risk of over-assertive outputs.

NOTE:

Additional project naming and prompt examples are provided in Prompts_and_Project_Names_Examples_v1.0.PDF (see orchestration/guides).

6.6 When to Use Mixed-Mode Projects

Mixed-mode projects are **advanced and optional**.

Use mixed-mode setup only when:

- one mode is clearly primary, and
- a secondary mode is needed for limited support.

Examples:

- Learning with operational perspective.
- Operational reasoning prepared for inspection.

Mixed-mode projects are defined fully in Chapter 8.

6.7 Stability Rule

Once a project is created:

- do not change the primary mode,
- do not swap or replace module files,
- do not alter the project purpose mid-session.

If posture needs to change:

- end the session, and
- create a new project.

Switch projects, not modes.

Key Principle

Project setup determines reasoning behavior.
Prompts only refine it.

---

7. Single-Mode Project Setup

Purpose of Single-Mode Projects

A single-mode project uses **one Access-Mode only**.

This is the **recommended default** for most use cases because it:

- reduces ambiguity,
- prevents reasoning drift,
- and produces more consistent outputs.

Single-mode projects are simpler to validate and easier to review.

7.1 Single-Mode Setup Principles

A correct single-mode project has:

- one primary Access-Mode,
- one full set of six modules from that mode,
- no secondary modes,
- no organizer_mode.md file, and
- a clear project title aligned with the mode.

If any of these elements are missing or mixed, the project is misconfigured.

7.2 ID_1 — Learning Project (Core Mode)

When to use ID_1

Use an ID_1 project when the goal is:

- learning,
- explanation,
- alignment of concepts,
- or building shared understanding.

This mode is **foundational** and avoids assumptions about operation or inspection.

Correct ID_1 setup

Project title example:

- *My AI Governance — Learning*

Files to upload:

- All six modules from `ID_1_core/modules/`

Files not to upload:

- No ID_2 or ID_3 modules
- No organizer_mode.md

Expected behavior

In a correct ID_1 project, the AI should:

- explain concepts clearly,
- define terms and scope,
- avoid claims about runtime behavior,
- avoid inspection or compliance language.

If the AI starts discussing enforcement, telemetry, or certification, mode drift has occurred.

7.3 ID_2 — Mentor Project (Operational Mode)

When to use ID_2

Use an ID_2 project when the goal is:

- operational reasoning,
- stress or failure analysis,
- scenario exploration,
- or mentoring through "what-if" thinking.

This mode assumes **use in context**, but not correctness or approval.

Correct ID_2 setup

Project title example:

- *My AI Governance — Mentor*

Files to upload:

- All six modules from `ID_2_access/modules/`

Files not to upload:

- No ID_1 or ID_3 modules
- No organizer_mode.md (unless mixed-mode is explicitly intended)

Expected behavior

In a correct ID_2 project, the AI should:

- discuss how governance behaves under change,
- explore uncertainty and edge cases,
- acknowledge limitations and risk,
- avoid claims of adequacy or compliance.

If the AI begins to "sign off" on solutions, validation discipline must be applied.

7.4 ID_3 — Organizer Project (Inspection Mode)

When to use ID_3

Use an ID_3 project when the goal is:

- organizing content,
- refining language,
- preparing material for review,
- or making outputs easier to inspect.

This mode focuses on **how content is read**, not how systems behave.

Correct ID_3 setup

Project title example:

- *My AI Governance — Organizer*

Files to upload:

- All six modules from `ID_3_operator/modules/`

Files not to upload:

- No ID_1 or ID_2 modules
- No organizer_mode.md (single-mode only)

Expected behavior

In a correct ID_3 project, the AI should:

- tighten wording,
- clarify structure,
- list assumptions explicitly,
- avoid operational or performance conclusions.

If the AI begins simulating runtime behavior, mode drift has occurred.

7.5 How to Detect Single-Mode Drift

Common signs of drift include:

- mixed terminology,
- sudden increase in certainty,
- conclusions outside the mode's intent,
- references to concepts not present in the uploaded modules.

When drift occurs:

- pause,
- restate the project purpose,
- or reset the session.

Key Principle

Single-mode projects are stable by design.
When in doubt, use one mode only.

---

8. Mixed-Mode Project Setup and `organizer_mode.md`

Terminology note:
In this guide, "mixed-mode" refers to projects that include one primary Access-Mode and one explicitly constrained secondary mode. The term is used to highlight the need for additional controls; secondary modes are never equal to, or interchangeable with, the primary mode.

Purpose of Mixed-Mode Projects

Mixed-mode projects allow a **secondary reasoning posture** to support a **single primary posture**.

They are used when:

- one mode must remain authoritative, and
- another mode provides limited perspective (not authority).

Mixed-mode projects are **advanced** and should be used sparingly.

8.1 When Mixed-Mode Is Appropriate

Use mixed-mode projects only when one of the following applies:

- Learning needs limited operational perspective
- Operational reasoning needs inspection-ready framing
- Learning needs reviewer-oriented clarity

If the goal can be achieved with a single mode, **do not** use mixed-mode.

8.2 Core Rule (Non-Negotiable)

A mixed-mode project must have:

- **one primary Access-Mode**, and
- **zero or more explicitly limited secondary modes**.

There is **no concept of equal modes**.

If primary and secondary roles are not explicit, the project is invalid.

8.3 File Requirements (Mixed-Mode)

For a mixed-mode project, upload **exactly**:

- **1 mode-specific README file (primary mode),**
- **6 modules from the primary mode**, and
- **6 modules from the secondary mode**, and
- **1 control file: `organizer_mode.md`**.

Total files:

- **14 files**.

Rules:

- Do not upload canonical `/modules` files.
- Do not upload partial sets.
- Do not upload more than one secondary mode.

8.4 `organizer_mode.md` (Mandatory Control File)

The file `organizer_mode.md` is **required** for all mixed-mode projects.

It acts as a **hard control** over reasoning posture.

If this file is missing:

- treat the project as **single-mode only**.

If this file is unclear or contradictory:

- treat all outputs as unreliable.

8.5 What `organizer_mode.md` Must Declare

The file must clearly state:

- which Access-Mode is **primary**,
- which Access-Mode (if any) is **secondary**, and
- what the secondary mode is **allowed to do**.

The primary mode always:

- determines assumptions,
- controls conclusions,
- and defines certainty.

Secondary modes:

- assist,
- clarify,
- or reframe,
- but do not decide.

8.6 Approved Mixed-Mode Patterns

The following patterns are permitted:

ID_1 Primary + ID_2 Secondary

Learning with operational perspective

Secondary mode may:

- identify stress points,
- highlight where assumptions may fail.

Secondary mode may not:

- assert runtime adequacy,
- conclude enforcement success.

ID_1 Primary + ID_3 Secondary

Learning with inspection clarity

Secondary mode may:

- tighten language,
- reduce ambiguity.

Secondary mode may not:

- assert standards alignment,
- imply approval or certification.

ID_2 Primary + ID_3 Secondary

Operational reasoning with inspection framing

Secondary mode may:

- organize findings,
- prepare review notes.

Secondary mode may not:

- reinterpret runtime behavior,
- downgrade uncertainty.

8.7 Prohibited Mixed-Mode Patterns

The following are **not allowed**:

- more than one secondary mode,
- switching primary mode mid-session,
- treating secondary output as authoritative,
- using ID_3 to justify ID_2 conclusions,
- introducing compliance or certification language.

If any of these occur, the session must be stopped.

8.8 Project Naming for Mixed-Mode

Project names must reflect the **primary mode first**.

Recommended patterns:

- *My AI Governance — Learning (with Runtime Lens)*
- *My AI Governance — Learning (with Review Lens)*
- *My AI Governance — Mentor (with Review Lens)*

Avoid names that suggest equivalence between modes.

8.9 How to Operate a Mixed-Mode Session

During a session:

- restate the primary mode when starting,
- allow secondary mode input only when relevant,
- record when secondary reasoning is applied,
- validate outputs as described in Chapter 5.

If posture begins to blur:

- pause,
- restate the rules,
- or end the session.

Key Principle

Mixed-mode is a controlled exception.
One mode leads. One mode supports.

---

9. Project Opening Instructions (Starter Prompts)

Purpose of Opening Instructions

Opening instructions are the **first message** used after a project is created and files are uploaded.

They:

- set the reasoning posture,
- reinforce the primary Access-Mode,
- and reduce early drift.

Opening instructions should be:

- short,
- explicit,
- and repeated if posture slips.

9.1 Learning Project — ID_1 (Core Mode)

Use this for **single-mode learning projects**.

Starter instruction:

You are operating in **Access-Mode ID_1 (Learning)**.
Explain concepts clearly using only the uploaded modules.
Do not assume runtime behavior, enforcement, inspection, or compliance.
Focus on definitions, scope, and understanding.
If a question belongs to another mode, say so.

9.2 Mentor Project — ID_2 (Operational Mode)

Use this for **single-mode mentoring or scenario reasoning projects**.

Starter instruction:

You are operating in **Access-Mode ID_2 (Operational)**.
Reason about governance under use, change, stress, or failure.
Use the uploaded modules as your source.
Do not assert adequacy, approval, or compliance.
Treat all conclusions as exploratory.

9.3 Organizer Project — ID_3 (Inspection Mode)

Use this for **single-mode organizing or review projects**.

Starter instruction:

You are operating in **Access-Mode ID_3 (Organizer)**.
Organize, clarify, and reframe content for review.
Focus on structure, wording, and assumptions.
Do not simulate runtime behavior or make approvals.
Keep language neutral and inspectable.

9.4 Mixed-Mode Projects (Reminder)

For mixed-mode projects:

- these starter instructions **must align** with `organizer_mode.md`,
- the **primary mode wording must come first**, and
- secondary modes must **not** be implied here.

If mixed-mode rules are not clear:

- stop,
- review `organizer_mode.md`,
- and restate the instruction.

When to Re-Use an Opening Instruction

Repeat the opening instruction if:

- the AI begins to drift,
- responses feel over-confident,
- or language no longer matches the mode.

Restating posture is a **correction**, not a failure.

Key Principle

The first instruction sets the tone.
Repeating it restores discipline.

---

10. Working Prompt Examples (Reference)

Purpose of This Chapter

This chapter provides **example prompts** that can be used **after** a project is correctly set up.

These prompts:

- assume the project is already configured (Chapters 6–8),
- respect the primary Access-Mode,
- and support consistent, reviewable outputs.

They are **examples**, not templates.
Rewrite them in your own words as needed.

10.1 ID_1 — Learning Prompts (Core Mode)

Use these only in **ID_1 single-mode projects**.

1. Explain this concept in plain language using the uploaded modules.
2. Define the key terms used here and explain how they relate.
3. What problem is this framework trying to solve?
4. What assumptions are stated explicitly in this section?
5. What is clearly out of scope?
6. Summarize this section without adding interpretation.
7. Rewrite this explanation for someone new to AI governance.
8. What would be misunderstood if this text were read quickly?

Reminder:
Do not introduce runtime behavior or inspection language.

10.2 ID_2 — Mentor Prompts (Operational Mode)

Use these only in **ID_2 single-mode projects**.

1. What happens if this assumption fails in operation?
2. Describe a realistic stress or edge case.
3. Where could drift occur over time?
4. What human oversight would be required here?
5. Which risks remain unresolved?
6. What signals would suggest governance controls are weakening?

7. How might misuse occur unintentionally?
8. What trade-offs are being made?

Reminder:
Do not conclude adequacy or approval.

10.3 ID_3 — Organizer Prompts (Inspection Mode)

Use these only in **ID_3 single-mode projects**.

1. Reorganize this content for clarity and review.
2. List all assumptions explicitly.
3. Remove ambiguous language without changing meaning.
4. Rewrite this for a neutral reviewer.
5. Identify statements that could be misinterpreted.
6. Separate facts from interpretation.
7. Convert this into a checklist or outline.
8. Tighten language while keeping it neutral.

Reminder:
Do not simulate runtime behavior or justify outcomes.

10.4 Mixed-Mode Prompts (Controlled Use)

Use these **only** in mixed-mode projects with a valid `organizer_mode.md`.

ID_1 Primary → ID_2 Secondary

(Learning with operational perspective)

- Explain the concept, then note where it might fail under stress.
- Identify operational risks without changing the explanation.

ID_1 Primary → ID_3 Secondary

(Learning with inspection clarity)

- Explain the concept, then rewrite it for reviewer clarity.
- Reduce ambiguity without adding standards claims.

ID_2 Primary → ID_3 Secondary

(Operational reasoning with inspection framing)

- Describe the operational scenario, then organize it for review.
- Prepare notes without reinterpreting behavior.

Rule:
Secondary prompts assist only. They do not override the primary mode.

10.5 Prompts to Avoid (All Modes)

Avoid prompts that:

- ask the AI to “decide,” “approve,” or “validate,”
- imply compliance or certification,
- request final judgments without human review,
- mix learning, operation, and inspection in one request.

If a prompt feels unclear, simplify it or restate the opening instruction.

10.6 Using This Chapter Safely

- Start with **one prompt at a time**.
- Validate outputs using Chapter 5.
- Record issues and corrections.
- Switch projects if posture needs to change.

Key Principle

Prompts refine behavior.
Project setup controls it.